

Cloud Health Office & Cloud Dental Office — Repository Assessment & Strategic Restructuring Plan

Prepared for: Mark — Aureliaware, Inc. **Date:** February 16, 2026

Executive Summary

Cloud Dental Office (CDO) is architecturally clean: 147 files, clear microservices boundaries, single-language (.NET 8), well-organized `src/Services/Shared/infrastructure/scripts` tree. It reads like a commercial product.

Cloud Health Office (CHO) has extraordinary depth — 933 files spanning 16 microservices, complete FHIR R4 APIs (Patient Access, Provider Access, Payer-to-Payer, Prior Auth), full X12 EDI pipeline, claims scrubbing engine, Argo workflows, and multi-tenant infrastructure — but the repo structure has accumulated significant organizational debt that obscures its commercial value and makes it harder for evaluators, adopters, and contributors to take it seriously.

The core problem: CHO's repo root looks like a developer's working folder, not a product. There are 56 markdown files at root, 15+ test artifacts scattered in the root directory, XSD schemas loose at the top level, a temp file (`tmp_x217.txt`), and no clear entry path for new users. Meanwhile the FHIR APIs — the single most commercially valuable component for CMS-0057-F compliance — have no OpenAPI specs, no quickstart, and are buried in `src/fhir/`.

The strategic opportunity is huge: CHO's claims scrubbing engine is exactly the "provider pre-edits claims before sending to payer" story that providers will pay for. Combined with the FHIR APIs and CDO as the provider-side product, you have a complete provider ↔ payer ecosystem that no other open-source project offers.

Part 1: Comparative Assessment

CDO — What's Working (The Gold Standard)

Attribute	Assessment
Root cleanliness	8 files at root: sln, license, readme, docker-compose, props — textbook
Project structure	Clean <code>src/Services/Shared/infrastructure/scripts</code> hierarchy
README quality	Architecture diagram, service table, quick start, tech stack, roadmap — professional
Language consistency	100% .NET 8 / C# — no language mixing
Microservices	6 bounded contexts + API Gateway + Portal — properly separated
Docker Compose	Complete orchestration with dependency ordering
K8s	Kustomize-based, clean namespace/apps/ingress split
Tests	Dedicated test project with tenant isolation tests
Shared libraries	Contracts + EdiCommon — proper bounded context communication

CHO — Current State Issues

1. Root Directory Sprawl (Critical)

56 markdown files at root. This is the #1 credibility killer. Commercial products have: README, LICENSE, CONTRIBUTING, CHANGELOG, SECURITY at root. Everything else goes in `/docs`.

The 56 files include overlapping/redundant docs:

- 8 deployment-related docs (DEPLOYMENT.md is 102KB alone)
- 4 security docs
- 16 implementation/summary docs (one-time "done" reports that should be in a changelog or wiki)
- 4 roadmap/launch docs (ROADMAP.md, ROADMAP-2026.md, V4-LAUNCH-ROADMAP.md, SAAS-LAUNCH-READINESS.md)

2. Test Artifacts in Root (Critical)

Loose test files polluting root:

- `test-*.json` (7 files) — payload fixtures
- `test-*.sh` (5 files) — test scripts
- `test-*.edi` (6 files) — EDI sample files
- `tmp_x217.txt` — temp file committed to repo
- `testing-status-report.md`, `test-plan-trading-partners.md`

3. Script Sprawl in Root (Major)

Setup/config scripts directly in root instead of `/scripts` :

- `bootstrap_repo.ps1`, `fix_repo_structure.ps1`
- `setup-integration-account.ps1`, `setup-portal-azuread-secret.sh`, `setup-stripe.sh`
- `configure-hipaa-trading-partners.ps1`, `configure-x12-agreements.ps1`
- `deploy-*.ps1`, `deploy-*.sh`
- `check-workflow-status.sh`, `validate-github-secrets.sh`

4. XSD Schemas in Root (Minor)

- `Companion_275AttachmentEnvelope.xsd`, `X12_005010X212_277.xsd`,
`X12_005010X217_278.xsd`

5. Mixed Language Architecture

- 64 TypeScript files (FHIR APIs, containers, scrubbing service, generators)
- 129 C# files (microservices)
- 24 Python files (test utilities, seed scripts)
- 35 Shell scripts + 16 PowerShell scripts

This isn't inherently wrong — it's a polyglot platform — but there's no architectural document explaining *why* each language is used where.

6. Missing Solution/Workspace File

CDO has `CloudDentalOffice.sln`. CHO has no equivalent top-level solution file or workspace config for the .NET services.

7. No Docker Compose

CDO has a complete docker-compose.yml. CHO has none — despite having 16+ services.

8. No OpenAPI Specifications

The FHIR APIs are well-implemented TypeScript classes but have:

- No OpenAPI/Swagger specs
- No published API docs
- No Postman collection
- No try-it-yourself quickstart

This is the single biggest gap for CMS-0057-F credibility.

9. Inconsistent Service Structure

Some services have Helm charts (eligibility-service), some have bare k8s manifests, some have neither. Some services have `Services/` and `Repositories/` folders with interfaces, others don't.

Part 2: CHO Restructuring Plan

Target Structure (Modeled After CDO's Clean Architecture)

```
cloudhealthoffice/
├── README.md                # Concise, professional (like CDO)
├── LICENSE
├── NOTICE
├── CONTRIBUTING.md
├── CHANGELOG.md
├── SECURITY.md
├── CODE_OF_CONDUCT.md
├── package.json
├── tsconfig.json
├── Directory.Build.props
├── docker-compose.yml      # NEW: Full stack orchestration
├──
├── docs/                  # ALL documentation consolidated here
│   └── architecture/
```

- | | | └─ ARCHITECTURE.md
- | | | └─ BRANCHING-STRATEGY.md
- | | | └─ decisions/ # ADRs (architecture decision records)
- | | └─ deployment/
 - | | | └─ DEPLOYMENT.md # Single consolidated guide
 - | | | └─ COSMOS-DB.md
 - | | | └─ GITHUB-ACTIONS.md
 - | | | └─ SECRETS-SETUP.md
- | | └─ security/
 - | | | └─ SECURITY-HARDENING.md
 - | | | └─ HIPAA-GUIDE.md
 - | | | └─ TOKEN-CONFIGURATION.md
- | | └─ compliance/
 - | | | └─ CMS-0057-F.md # NEW: Regulation-focused guide
 - | | | └─ FHIR-IMPLEMENTATION.md
 - | | | └─ EDI-TRANSACTIONS.md
- | | └─ onboarding/
 - | | | └─ QUICKSTART.md
 - | | | └─ ONBOARDING.md
 - | | | └─ TROUBLESHOOTING.md
- | | └─ roadmap/
 - | | | └─ ROADMAP.md # Single consolidated roadmap
 - | | | └─ RELEASE-NOTES.md
- | | └─ internal/ # Historical/one-time implementation notes
 - | | | └─ 276-277-implementation.md
 - | | | └─ prior-auth-implementation.md
 - | | | └─ ... # Move all *-SUMMARY.md, *-COMPLETE.md her
- | └─ api/ # NEW: OpenAPI specs & quickstarts
 - | | └─ openapi/
 - | | | └─ patient-access-api.yaml # OpenAPI 3.1 specs
 - | | | └─ provider-access-api.yaml
 - | | | └─ payer-to-payer-api.yaml
 - | | | └─ prior-auth-api.yaml
 - | | | └─ claims-scrubbing-api.yaml
 - | | └─ postman/
 - | | | └─ CloudHealthOffice.postman_collection.json
 - | | └─ quickstarts/
 - | | | └─ patient-access-quickstart.md
 - | | | └─ provider-access-quickstart.md
 - | | | └─ claims-scrubbing-quickstart.md
 - | | | └─ cms-0057f-compliance-quickstart.md
- | └─ src/
 - | | └─ fhir/ # Existing FHIR implementations
 - | | └─ ai/ # AI resolution engine
 - | | └─ security/ # HIPAA security module

```

|   ├── logicapps/                # Azure Logic Apps
|   └── onboarding/              # Tenant onboarding
|
|── services/                    # .NET microservices (existing)
|   ├── appeals-service/
|   ├── attachment-service/
|   ├── authorization-service/
|   ├── benefit-plan-service/
|   ├── claims-scrubbing-service/
|   ├── claims-service/
|   ├── coverage-service/
|   ├── eligibility-service/
|   ├── enrollment-import-service/
|   ├── member-service/
|   ├── payment-service/
|   ├── provider-service/
|   ├── reference-data-service/
|   ├── shared/
|   ├── sponsor-service/
|   ├── tenant-service/
|   └── trading-partner-service/
|
|── containers/                  # Container images (existing)
|
|── infrastructure/              # IaC consolidated
|   ├── argo-events/             # Move from /argo-events
|   ├── argo-workflows/          # Move from /argo-workflows
|   ├── k8s/
|   ├── azure/
|   │   ├── azuredeploy.json     # Move from root
|   │   ├── attachments-logicapps/ # Move from /attachments_logicapps_package
|   │   └── bicep/
|   └── docker/
|
|── schemas/                     # X12 & FHIR schemas
|   ├── x12/
|   │   ├── Companion_275AttachmentEnvelope.xsd
|   │   ├── X12_005010X212_277.xsd
|   │   └── X12_005010X217_278.xsd
|   └── fhir/
|       └── ...
|
|── scripts/                     # ALL scripts consolidated (existing + roo
|   ├── setup/                   # bootstrap, configure, deploy scripts
|   ├── testing/                 # test scripts
|   ├── utils/                   # existing utils
|   └── seeds/

```

```

|
├─ tests/                                # ALL test artifacts consolidated
|   └─ fixtures/
|       └─ edi/                          # Move test-x12-*.edi files here
|       └─ json/                         # Move test-*.json files here
|       └─ ...
|   └─ unit/
|   └─ integration/
|   └─ e2e/
|
├─ site/                                # Marketing website (existing)
|
├─ tools/                               # Migration wizard etc. (existing)
|
└─ config/                             # Configuration schemas & examples (existing)

```

File Movement Summary

Action	Count	From → To
Move root .md to /docs	~49 files	Root → docs/ subdirectories
Move test-*.json to /tests/fixtures/json	7 files	Root → tests/fixtures/json/
Move test-*.sh to /scripts/testing	5 files	Root → scripts/testing/
Move test-*.edi to /tests/fixtures/edi	6 files	Root → tests/fixtures/edi/
Move *.xsd to /schemas/x12	3 files	Root → schemas/x12/
Move root scripts to /scripts/setup	~10 files	Root → scripts/setup/
Move argo-events/	directory	Root → infrastructure/argo-events/
Move argo-workflows/	directory	Root → infrastructure/argo-workflows/
Move azuredeploy.json	1 file	Root → infrastructure/azure/
Move attachments_logicapps_package/	directory	Root → infrastructure/azure/

Delete tmp_x217.txt	1 file	Remove
Delete README.md.bak	1 file	Remove

Part 3: OpenAPI Specifications & FHIR Quickstarts

Why This Matters

CMS-0057-F requires health plans to implement Patient Access, Provider Access, and Payer-to-Payer Data Exchange APIs by January 1, 2027. CHO already has the implementations. Publishing OpenAPI specs turns this from “code you have to read” to “API you can try in 5 minutes.”

OpenAPI Specs to Create

Based on the existing FHIR API implementations in `src/fhir/`:

1. Patient Access API (`patient-access-api.yaml`)

- `GET /fhir/Patient/{id}` — Patient demographics
- `GET /fhir/ExplanationOfBenefit` — EOBs (claims history)
- `GET /fhir/Claim` — Claims
- `GET /fhir/Encounter` — Encounters
- `GET /fhir/CoverageEligibilityResponse` — Coverage/eligibility
- OAuth 2.0 patient consent flow
- Based on: Da Vinci PDex IG, US Core 3.1.1+

2. Provider Access API (`provider-access-api.yaml`)

- `GET /fhir/Patient` — Active patient roster
- `GET /fhir/ExplanationOfBenefit` — Claims/remittance data
- `GET /fhir/Condition` — Diagnoses
- `GET /fhir/Procedure` — Procedures
- `POST /fhir/$bulk-member-match` — Bulk member matching
- Based on: Da Vinci PDex IG, Provider Access requirements

3. **Payer-to-Payer API** (payer-to-payer-api.yaml)

- POST /fhir/\$member-match — **Member matching**
- POST /fhir/\$everything — **Complete member data transfer**
- GET /fhir/Bundle — **Paginated data retrieval**
- **Based on: CMS-0057-F Payer-to-Payer requirements**

4. **Prior Authorization API** (prior-auth-api.yaml)

- POST /fhir/Claim — **Submit prior auth**
- GET /fhir/ClaimResponse/{id} — **Check auth status**
- POST /cds-services/prior-auth — **CDS Hooks integration**
- **X12 278 ↔ FHIR bidirectional mapping**
- **Based on: Da Vinci PAS IG**

5. **Claims Scrubbing API** (claims-scrubbing-api.yaml)

- POST /api/v1/claims/validate — **Single claim validation**
- POST /api/v1/claims/batch-validate — **Batch validation**
- GET /api/v1/rules — **List active rules**
- POST /api/v1/rules/custom — **Add custom rules**
- GET /api/v1/metrics — **First-pass rates and analytics**

FHIR Quickstarts to Create

1. **CMS-0057-F Compliance Quickstart** — “Verify your FHIR compliance in 15 minutes”
2. **Patient Access Quickstart** — “Expose patient data via FHIR in 30 minutes”
3. **Provider Access Quickstart** — “Give providers access to member data”
4. **Claims Scrubbing Quickstart** — “Pre-edit claims before payer submission”

Part 4: Strategic Initiatives

Initiative 1: Provider Pre-Edit / Claims Scrubbing as a Service

This is CHO's killer feature for providers. The claims scrubbing service in CHO already implements:

- 30+ standard validation rules (data completeness, code validation, date logic, amount logic, modifier validation)
- Custom rule support per payer
- Pre-adjudication scoring
- First-pass rate analytics

The provider value proposition: "See what edits the payer will require *before* you submit the claim. Fix issues at the point of care, not after a 30-day denial cycle."

Strategic packaging:

- Expose the claims scrubbing API as a standalone SaaS offering
- CDO integrates natively (dental claims → CHO scrubbing → clearinghouse)
- Any EHR/PMS can call the API via REST
- Pricing: per-claim-scrubbed model (pennies per claim, scales with volume)
- Differentiation: "Open-source rule engine, no black box"

Initiative 2: CDO + CHO Ecosystem Play

Position CDO as the first reference implementation of a provider system that natively connects to CHO's payer platform. This is unique in the market — no other open-source project offers both sides of the healthcare data exchange.

- CDO generates 837D → CHO scrubs → CHO routes to payer
- Payer responds 277/835 → CHO translates → CDO auto-posts ERA
- Patient requests data → CHO FHIR Patient Access API → compliant response

Initiative 3: CMS-0057-F Compliance Accelerator

Target audience: Small-to-mid health plans that can't afford \$5M+ vendor implementations (Cognizant/TriZetto, Inovalon, etc.) but face the January 2027 deadline.

Deliverables:

- Published OpenAPI specs proving API coverage
- Compliance checker tool (already exists in `src/fhir/compliance-checker.ts`)
- Quickstart guides with synthetic data
- Whitepaper: "CMS-0057-F Compliance with Open-Source: A 90-Day Implementation Guide"

Initiative 4: SaaS Platform User Acquisition

For CDO (Provider Market):

- Target: Independent dental practices (Matthew's practice as reference implementation)
- Free tier: 1 provider, 500 patients, basic claims
- Paid tier: Multi-location/DSO, real-time eligibility, ERA auto-posting
- Channel: Dental conferences, DSO associations, dental software forums

For CHO (Payer Market):

- Target: Small Medicaid MCOs, TPAs, startup health plans
 - Free tier: Development/sandbox environment
 - Paid tier: Production multi-tenant, compliance certification support
 - Channel: AHIP conferences, state Medicaid RFPs, compliance consulting partners
-

Part 5: Prioritized Action Items

Phase 1: Repository Cleanup (Week 1-2)

1. ☐ Create `/docs` directory structure and move 49 root markdown files
2. ☐ Move test fixtures to `/tests/fixtures/`
3. ☐ Move root scripts to `/scripts/setup/`
4. ☐ Move XSD schemas to `/schemas/x12/`
5. ☐ Move argo-events/ and argo-workflows/ to `/infrastructure/`
6. ☐ Delete `tmp_x217.txt` and `README.md.bak`

7. ☐ Consolidate deployment docs into single guide
8. ☐ Write new concise README.md (modeled on CDO)
9. ☐ Add docker-compose.yml for local dev
10. ☐ Add .NET solution file for services

Phase 2: API Documentation (Week 2-4)

11. ☐ Create OpenAPI 3.1 spec for Patient Access API
12. ☐ Create OpenAPI 3.1 spec for Provider Access API
13. ☐ Create OpenAPI 3.1 spec for Payer-to-Payer API
14. ☐ Create OpenAPI 3.1 spec for Prior Auth API
15. ☐ Create OpenAPI 3.1 spec for Claims Scrubbing API
16. ☐ Create Postman collection
17. ☐ Write CMS-0057-F compliance quickstart
18. ☐ Write Patient Access quickstart with synthetic data
19. ☐ Write Claims Scrubbing quickstart

Phase 3: Claims Scrubbing as a Service (Week 3-6)

20. ☐ Extract claims scrubbing service as standalone deployable
21. ☐ Build REST API wrapper (if not already Express/Fastify)
22. ☐ Create CDO integration: Portal → CHO Claims Scrubbing API
23. ☐ Build provider-facing dashboard showing edit suggestions
24. ☐ Add Stripe metered billing for per-claim pricing

Phase 4: SaaS Launch Prep (Week 4-8)

25. ☐ CDO: Finalize Stripe signup flow
26. ☐ CDO: Deploy to production (DigitalOcean DOKS or Azure)
27. ☐ CHO: Deploy sandbox environment for evaluators
28. ☐ Build landing pages emphasizing CMS-0057-F deadline
29. ☐ Create demo video showing CDO → CHO claims flow

Appendix: CDO vs CHO Metrics

Metric	CDO	CHO
Total files	147	933
Root markdown files	1 (README)	56
Root test artifacts	0	18
Root scripts	0	10+
Languages	C# only	C#, TypeScript, Python, Shell, PowerShell
Services	6 + Gateway + Portal	16 + containers + workflows
Docker Compose	✓	✗
Solution file	✓	✗
OpenAPI specs	✗ (Swagger via .NET)	✗
README quality	A	B- (comprehensive but overwhelming)
K8s	Kustomize	Mixed (Helm, bare YAML, Argo)
CI/CD	1 workflow	15 workflows
Test organization	Clean test project	Scattered across root + /tests
Repository clones	New	40,000+

This assessment was generated from direct analysis of both repository codebases. The restructuring plan preserves all existing code and functionality while reorganizing for commercial-grade presentation.